

Task 5 — Audit a Skill Before Trusting It

Lecture 8: Skills & Connectors — Teach AI Once, Connect It to Your Apps

Student	Abdul Manan
Program	Panaversity — The AI Agent Factory (AI-101)
Skill audited	find-skills (vercel-labs/agent-skills)
AI tool used	Claude (web interface)
Install method	SKILL.md uploaded manually into Claude
Date	July 2026

1. Project Title and What It Does

Skill Safety Audit — find-skills

This project is a safety review of a third-party skill I did not write, carried out before deciding whether to trust it. The goal of Task 5 is not to build anything — it is to build the *habit* of vetting a skill before enabling it, and to prove I can say in plain English what a skill does and whether it is safe, without reading a single line of code.

The skill I audited is `find-skills`, a discovery helper from Vercel Labs. It does no real work of its own. Instead it teaches the AI how to find *other* skills from the open agent-skills ecosystem (`skills.sh`) and install them through the `npm` `skills` CLI. It is a shop assistant for the skills store: it does not build anything itself, it helps you find and install things other people built.

2. Which AI Tool(s) and App(s) I Used

Component	What I used
AI tool	Claude (web) — used to read the SKILL.md and produce the plain-English explanation
Skill source	vercel-labs/agent-skills — the <code>find-skills</code> package
Directory	skills.sh — the official open skills directory referenced by the skill
Install route	Manual upload of the SKILL.md file directly into Claude
Apps connected	None. This task is read-only inspection; no connector or account access was involved.

Note on method: rather than installing through the CLI, I uploaded the raw `SKILL.md` into Claude. This was deliberate — it let me read the exact instruction text *before* anything executed on my machine, which is the safer order of operations for an audit.

3. The Prompts I Used (and How I Refined Them)

Initial prompt

```
Read this skill and tell me what it does.
```

This was too loose. It invited a summary, and a summary is exactly what a malicious skill would want me to settle for — a friendly gloss instead of the actual instruction body. It also gave me no safety analysis at all.

Refined prompt (the one I used)

```
Read the skill I just installed and tell me, in plain language, exactly what it instructs you to do. Then flag anything in it that contacts an external server, handles passwords or credentials, or could send my data somewhere I didn't intend. If it's clean, say so plainly.
```

What the refinement changed, and why it mattered

- **"exactly what it instructs you to do"** — forces a walk through the actual instruction body, not the self-description in the frontmatter. A skill's own description is marketing copy; the instructions are the contract.
- **Naming the three risk categories explicitly** — external server, credentials, unintended data egress. These are the exact three checks Task 5 asks for. Naming them stops the AI from picking its own definition of "sensitive."
- **"If it's clean, say so plainly"** — this one matters more than it looks. Without it, an AI will hedge and manufacture concerns to appear thorough, which trains me to ignore warnings. Explicitly permitting a clean verdict makes the

flagged items meaningful when they do appear.

4. What the Skill Instructs the AI to Do — In Plain English

The skill activates whenever I ask something like "how do I do X", "is there a skill for X", or "can you do X" — anything that sounds like I want a capability that might already exist as a packaged skill. Once triggered, it follows six steps:

1. **Understand the need** — identify the domain (React, testing, deployment, and so on) and the specific task, then judge whether a skill plausibly exists for it.
2. **Check the leaderboard first** — look at the skills.sh homepage, which ranks skills by total installs, before running any search. This surfaces the popular, battle-tested options first.
3. **Search the CLI** — if the leaderboard does not cover it, run `npx skills find <query>` with keywords derived from my request.
4. **Verify quality before recommending** — the skill explicitly states *do not recommend based on search results alone*, and sets three checks: install count (prefer 1K+, be cautious under 100), source reputation (official orgs such as vercel-labs, anthropics, microsoft over unknown authors), and GitHub stars on the source repo (under 100 stars means treat with skepticism).
5. **Present options to me** — the skill name, what it does, install count, source, the install command, and a link to skills.sh.
6. **Offer to install** — if I agree, run `npx skills add <owner/repo@skill> -g -y`.

It also carries a reference table of common categories (web development, testing, DevOps, documentation, code quality, design, productivity), search tips such as using specific keywords and trying synonyms, and a fallback path: if no skill is found, admit it, offer to help directly, and suggest `npx skills init` so I can create my own.

5. Safety Assessment — The Three Flags

Does it handle credentials or passwords?

No. There is no mention of tokens, API keys, OAuth, passwords, logins, or `.env` files anywhere in the file. It never asks me for a secret and never reads one.

Does it send my data somewhere I didn't intend?

No. Nothing in the skill reads my files, conversation history, clipboard, or machine state and ships it out. The only thing that leaves my machine is the search query text I type — for example, "react performance" goes to the skills registry as a search term. That is expected and unavoidable for any search tool, and it is not data I did not intend to send.

Does it contact an external server?

Yes — and this is the entire point of the skill, not hidden behaviour. It reaches three destinations:

Destination	Why it is contacted
skills.sh	Browse the leaderboard and retrieve skill listings
npm registry	<code>npx</code> downloads and runs the <code>skills</code> CLI package
GitHub	The actual skill packages are installed from GitHub repositories

This is exactly what a package manager does. The trust model is the same as `npm install` or `pip install`. Nothing is disguised or undisclosed.

The one thing genuinely worth flagging

Step 6 instructs the AI to run:

```
npx skills add <owner/repo@skill> -g -y
```

Two details in that command deserve attention:

- `-g` installs the skill **globally** at user level — so it becomes available in every project, not just the current one.
- `-y` **skips the confirmation prompt** — the CLI will not stop and ask "are you sure?" before installing.

The consequence: the safety gate is *not* the CLI double-checking me. The gate is (1) the AI's own vetting from Step 4, and (2) my "yes, install it." After that, third-party code from a GitHub repository lands in my environment with no

second prompt. This is standard supply-chain risk for any package manager, but it is worth being conscious of rather than clicking through on autopilot.

6. How I Tested and Verified It Worked

- I uploaded the raw `SKILL.md` into Claude and asked it to read the file and explain, in plain language, exactly what it instructs it to do — then to flag the three risk categories.
- I deliberately did **not** rely on the skill's own frontmatter description of itself. I had the full instruction body read through step by step, because the description is what the author wants me to believe and the body is what actually runs.
- I cross-checked the three network destinations against what each step actually calls: `npx` resolves through npm, `skills find` queries `skills.sh`, and `skills add` pulls from GitHub. The claimed egress matches the mechanical reality of the commands.
- I confirmed the negative findings by absence: I searched the file for any reference to credentials, tokens, environment variables, file reads, or upload behaviour. There are none.
- The entire audit was performed by reading plain English. No code literacy was required — which is itself the point Task 5 is making.

7. Decision and Verdict

Verdict: Safe to enable.

The skill does what it says, discloses its network access as an inherent function rather than a hidden one, handles no secrets, and exfiltrates nothing. I would enable it — with the conditions listed below.

Why

- **No credential handling.** Clean on that front entirely.
- **No exfiltration.** Nothing reads my data and sends it out. Search queries go out as search queries; that is the whole of it.
- **Network access is disclosed and inherent.** It contacts `skills.sh`, npm, and GitHub because that is literally what a package manager must do to function. Nothing is hidden or disguised as something else.
- **It actively encourages caution.** Step 4 tells the AI to vet install counts, source reputation, and GitHub stars *before* recommending anything. A malicious skill would not include a vetting step that makes its own recommendations harder to accept.
- **No trickery in the instructions.** No attempt to override my instructions, no "ignore previous rules", no silent behaviour, no obfuscation, no unexplained commands.

Conditions I am attaching to my own use

1. I will not treat the `-y` flag as permission to install carelessly. My "yes" is the confirmation prompt, so I will actually read what is being installed before giving it.
2. I will respect the skill's own advice: anything under 100 installs, from an unknown author, or from a repository with fewer than 100 stars gets skipped unless I have a specific reason.
3. The risk here is not `find-skills` itself — it is whatever `find-skills` helps me install next. This skill is a door, not a room. I audit the room too.

8. What Worked, What Did Not, and Problems Faced

What worked

- Uploading the `SKILL.md` manually rather than installing through the CLI turned out to be the right call. It let me inspect the instructions before anything ran, which is the correct order for an audit — read first, execute second.
- Naming the three risk categories directly in the prompt produced a structured answer that mapped cleanly onto the task requirements, instead of a vague "looks fine to me."
- The plain-English requirement held up. I genuinely did not need to read code to reach a defensible verdict, which was the point of the exercise.

What did not work at first

- My initial prompt ("tell me what it does") returned a summary that essentially paraphrased the skill's own description. That is close to useless for an audit — I was being told what the author claims, not what the file instructs. Refining the prompt to demand the instruction body was the fix.

- My first instinct was to treat "contacts an external server" as automatically a red flag. It is not. For a package manager, network access is the function. The real question is not *whether* it phones out but *where, why, and whether that is disclosed*. Learning to make that distinction was the most useful part of this task.

Problems faced

- **The clean-verdict trap.** There is real pressure in a security assignment to find something wrong, because a report that says "it's fine" feels like it did no work. I had to resist inventing concerns. The honest finding is that the file is clean, and the one genuine flag — the `-y` auto-confirm — is worth more than five manufactured ones.
- **Scope ambiguity.** It took me a moment to realise the audit boundary. The skill is safe; the ecosystem it opens the door to is a separate question with a separate answer. Conflating the two would have produced either a falsely alarming verdict or a falsely reassuring one.
- **Trusting the auditor.** I am using an AI to audit a file that instructs an AI. If the file contained a prompt injection, the auditor could in principle be the thing compromised. In this case the file is short enough to read myself as a cross-check, and I did — but this does not scale to a long or obfuscated skill, and that limitation is worth stating rather than hiding.

9. Takeaway

The main lesson from this task is that a skill can be completely clean and still be a gateway to risk. `find-skills` does nothing dangerous, but it *installs code from the internet with no second confirmation*. The audit is not finished when I decide the file is safe — it restarts every time this skill hands me a new one.

That is the habit Task 5 is actually training: not "is this file malicious", but "do I understand what I am agreeing to, every time I agree to it."